

Bluetooth Master Mode

Bluetooth Master Mode

1. BLE Overview
2. Core Concepts
 - 2.1 GAP — Generic Access Profile
 - 2.2 GATT — Generic Attribute Profile
 - 2.3 CCCD — Client Characteristic Configuration Descriptor
 - 2.4 Connection State Machine
 - 2.5 NimBLE Protocol Stack
3. ESP32-S3 BLE Hardware Features
4. Project Architecture and Flow
5. Hardware Dependencies
6. Source Code Analysis
 - 6.1 Header Files and Macro Configuration
 - 6.2 app_main() — Entry Point
 - 6.3 ble_central_init() — Initialization
 - 6.4 on_sync() — Sync Callback and start_scan()
 - 6.5 gap_event_cb() — GAP Event Callback
 - 6.6 GATT Discovery Chain — Callback Chain Calls
 - 6.7 vSendTask() — Periodic Send Task
 - 6.8 flush_timer_cb() — RX Accumulation Flush
7. Operation Flow
 - 7.1 Build and Flash Flow
 - 7.2 Testing Method 1: Dual-Board Test
 - 7.3 Testing Method 2: With Phone Bluetooth Debugging Assistant
 - 7.4 Normal Communication Log (Dual-Board Test)
 - 7.5 Edge Cases
8. Common Problems

1. BLE Overview

BLE (Bluetooth Low Energy) is a short-range wireless communication protocol designed for low power consumption and low bandwidth scenarios. Compared to classic Bluetooth, BLE connects faster (millisecond level) and consumes less power (coin battery can run for months).

Importance of BLE Master tutorial: BLE master (Central) plays the role of actively discovering and connecting to slaves, serving as the control end for data aggregation and command delivery. Phones, tablets, gateways and other central devices all run as master—mastering master programming means being able to build central nodes that actively scan, auto-connect, and read/write remote slave data. This is the other half of core capabilities for building complete BLE systems.

Typical application scenarios: Phone app scans nearby temperature sensors and reads data, industrial gateway periodically collects sensor values from multiple slave nodes, smart watch as master reads heart rate belt and running sensor data.

BLE role system:

Role	GAP Name	Responsibilities
Master	Central	Actively scans nearby devices, initiates connections, discovers services and characteristics, reads/writes data
Slave	Peripheral	Advertises its presence, accepts connections, provides GATT service for master access

This project implements **BLE Central (Master)**, responsible for scanning slave named "ESP-IDF", after connection automatically discovers GATT service, enables notifications, periodically writes data.

2. Core Concepts

2.1 GAP — Generic Access Profile

GAP (Generic Access Profile) defines how BLE devices **discover and connect** to each other.

GAP operations involved in this master:

Operation	API	Description
Scan	<code>ble_gap_disc()</code>	Actively scans nearby advertising, discovers slaves
Connect	<code>ble_gap_connect()</code>	Initiates connection request to target address
Disconnection reconnection	<code>gap_event_cb()</code>	Detects disconnection then automatically re-scans

2.2 GATT — Generic Attribute Profile

GATT (Generic Attribute Profile) defines how BLE devices **organize and interact with data**. Data is organized in **Service** → **Characteristic** → **Descriptor** three-level hierarchy.

This project's GATT custom service structure:

```
Service: 0xFFFF0 (Custom primary service)
├─ Characteristic: 0xFFFF1 (NOTIFY – Slave → Master reporting)
│   └─ Descriptor: 0x2902 (CCCD – Client Characteristic Configuration Descriptor)
└─ Characteristic: 0xFFFF2 (WRITE – Master → Slave delivery)
```

2.3 CCCD — Client Characteristic Configuration Descriptor

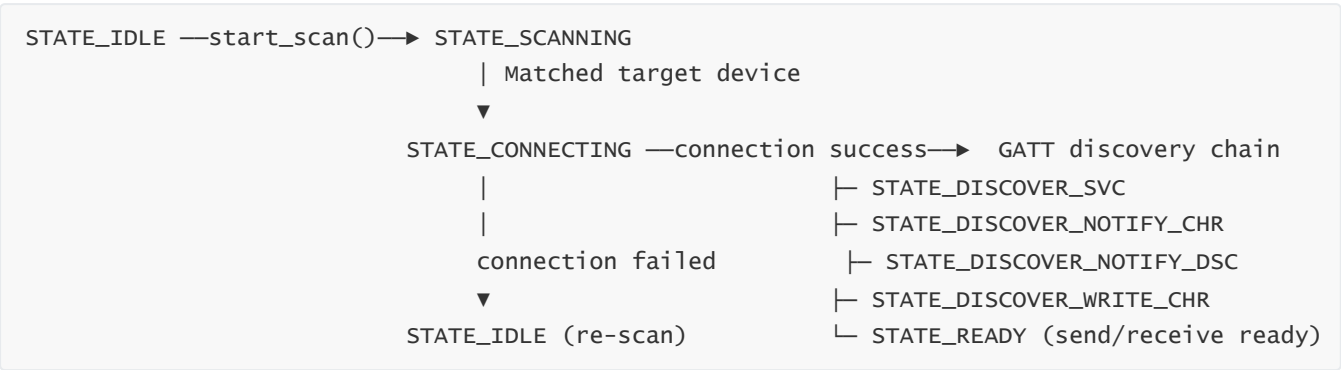
CCCD (Client Characteristic Configuration Descriptor) UUID is fixed at `0x2902`. Master writes CCCD to subscribe/unsubscribe to slave's notifications (Notify) or indications (Indicate).

Value	Meaning
0x0000	Disable notifications and indications
0x0001	Enable notifications (Notify)
0x0002	Enable indications (Indicate)

This project writes 0x0001 to enable slave's 0xFFFF1 notification.

2.4 Connection State Machine

Central device manages the entire lifecycle through a **state machine**, ensuring controllable behavior in async callbacks:



2.5 NimBLE Protocol Stack

ESP32-S3 uses **Apache NimBLE** as BLE Host protocol stack. NimBLE runs on FreeRTOS, in independent task `vnimbleHostTask`, driving all BLE operations through `nimble_port_run()` event loop. NimBLE's advantage is small code size (~32KB ROM) and low memory footprint.

3. ESP32-S3 BLE Hardware Features

Feature	Parameter
Wireless standard	BLE 5.0
Max TX power	+20 dBm
RX sensitivity	-97 dBm (1Mbps)
Max MTU	251 bytes (this project sets 256 negotiation upper limit)
Antenna	Onboard PCB antenna
Concurrent connections	Supported (this project single connection)
Protocol stack	NimBLE (Apache open source)

BLE and WiFi share 2.4GHz antenna, both can coexist but use time division multiplexing.

4. Project Architecture and Flow

```
app_main()
├─ nvs_flash_init()           ← Init NVS flash (stores pairing info)
├─ ble_central_init()        ← BLE Central overall initialization
│   ├─ nimble_port_init()    Init NimBLE protocol stack
│   ├─ ble_hs_cfg callback config Register sync/reset/store callbacks
│   ├─ ble_svc_gap_init()    GAP service init
│   ├─ ble_svc_gatt_init()   GATT service init
│   ├─ ble_att_set_preferred_mtu(256) Set max transmission unit
│   ├─ xSemaphoreCreateBinary() Create send sync semaphore
│   ├─ xTimerCreate(flush)    Create RX flush timer
│   ├─ xTaskCreate(vSendTask) Create periodic send task
│   └─ nimble_port_freertos_init() Start NimBLE host task
│       └─ on_sync()
│           └─ start_scan() ← Start BLE active scan

gap_event_cb() ← GAP event callback (scan/connect/disconnect/receive notify)
├─ BLE_GAP_EVENT_DISC Scan result → match "ESP-IDF" → initiate connection
├─ BLE_GAP_EVENT_CONNECT Connection success → MTU negotiate → discover_service()
├─ BLE_GAP_EVENT_MTU MTU negotiation complete
├─ BLE_GAP_EVENT_DISCONNECT Disconnected → clear state → start_scan()
└─ BLE_GAP_EVENT_NOTIFY_RX Receive notify → buffer + flush timer

GATT discovery chain (callback chain calls):
discover_service() Discover 0xFFFF0 service by UUID
→ discover_notify_characteristic() Discover 0xFFFF1 notify characteristic
→ discover_notify_cccd() Discover CCCD descriptor
→ enable_notifications() Write CCCD=0x0001 enable notify
→ discover_write_characteristic() Discover 0xFFFF2 write characteristic
→ mark_ready() Mark STATE_READY + release semaphore

vSendTask() ← Periodic send task
while(1):
├─ xSemaphoreTake(g_send_sem) Wait for semaphore (first=ready,
subsequent=timeout)
└─ ble_gattc_write_flat() Write "Hello from ESP32 Master" to 0xFFFF2
```

5. Hardware Dependencies

Hardware Connection: No external wiring, using ESP32-S3 onboard PCB antenna for BLE wireless communication.

Required components: `nvs_flash`, `bt` (NimBLE), `FreeRTOS`, `esp_log`

Companion device: Need another ESP32-S3 running BLE Slave routine as Peripheral, advertising name `"ESP-IDF"`.

6. Source Code Analysis

Full source: [Source Code](#) → [ESP-IDF](#) → [5.Network_Protocol](#) → [9.BLE](#) → [BLE_Master](#)

6.1 Header Files and Macro Configuration

Master macros are symmetric with slave—service UUID, notify/write characteristic UUIDs must match on both sides for normal communication. State machine enum `conn_state_t` defines all 8 states from idle to ready, ensuring controllable behavior in async callbacks. `g_send_sem` semaphore synchronizes GATT discovery completion with periodic send task; `g_target_addr` caches slave address to support disconnection reconnection.

```
#include <stdio.h>
#include <string.h>
#include "esp_log.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/timers.h"
#include "nimble/nimble_port.h"
#include "nimble/nimble_port_freertos.h"
#include "services/gap/ble_svc_gap.h"
#include "services/gatt/ble_svc_gatt.h"
#include "host/ble_hs.h"
#include "host/ble_gap.h"
#include "host/ble_gatt.h"

#define TAG                                "BLE_MASTER"
// Target peripheral name
#define TARGET_DEVICE_NAME                "ESP-IDF"
// Target service UUID
#define BLE_SVC_UUID                      0xFFFF0
// Notify characteristic UUID
#define BLE_NOTIFY_CHR_UUID              0xFFFF1
// Write characteristic UUID
#define BLE_WRITE_CHR_UUID               0xFFFF2
// Periodic send interval (ms)
#define SEND_INTERVAL_MS                 5000
// RX flush timeout (ms)
#define FLUSH_TIMEOUT_MS                 50

// Connection state machine
typedef enum {
    STATE_IDLE,
    STATE_SCANNING,
    STATE_CONNECTING,
    STATE_DISCOVER_SVC,
    STATE_DISCOVER_NOTIFY_CHR,
    STATE_DISCOVER_NOTIFY_DSC,
    STATE_DISCOVER_WRITE_CHR,
    STATE_READY,
} conn_state_t;
```

```

// Global state variables
static conn_state_t      g_state;
// Connection handle
static uint16_t          g_conn_hdl;
// Service handle range
static uint16_t          g_svc_start, g_svc_end;
// Characteristic handles
static uint16_t          g_notify_hdl, g_write_hdl;
// Send semaphore
static SemaphoreHandle_t g_send_sem;
// Receive buffer
static uint8_t           g_rx_data[512];
// Receive data length
static uint16_t          g_rx_len;
// Flush timer
static TimerHandle_t     g_flush_timer;
// Target device address
static ble_addr_t        g_target_addr;

```

6.2 app_main() — Entry Point

Init NVS flash to store BLE pairing info, then call `ble_central_init()` to one-click start master-side Bluetooth. If NVS partition abnormal (first flash or partition corrupted) erase and rebuild.

```

void app_main(void)
{
    // Init NVS flash
    esp_err_t ret = nvs_flash_init();
    // If NVS partition error
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES ||
        ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        // Erase and retry
        ESP_ERROR_CHECK(nvs_flash_erase());
        ESP_ERROR_CHECK(nvs_flash_init());
    }

    // Start BLE Central
    ble_central_init();
}

```

6.3 ble_central_init() — Initialization

`ble_central_init()` in order: init NimBLE protocol stack → register host callbacks → init GAP/GATT basic services → set MTU → create semaphore and timer → create send task → start host event loop.

`ble_att_set_preferred_mtu(256)` negotiates larger MTU to avoid fragmentation of long data; `xTimerCreate` creates 50ms timer for RX accumulation—BLE notifications may arrive in fragments, accumulate together for one complete message.

```

void ble_central_init(void)
{
    // Init NimBLE port

```

```

ESP_ERROR_CHECK(nimble_port_init());

// Register reset callback
ble_hs_cfg.reset_cb = on_reset;
// Register sync callback
ble_hs_cfg.sync_cb = on_sync;
// Register GATT callback
ble_hs_cfg.gatts_register_cb = gatt_svr_register_cb;
// Register store callback
ble_hs_cfg.store_status_cb = ble_store_util_status_rr;

// Set device name
ble_svc_gap_device_name_set("MASTER");
// Init GAP service
ble_svc_gap_init();
// Init GATT service
ble_svc_gatt_init();
// Set MTU to 256
ble_att_set_preferred_mtu(256);

// Create send semaphore
g_send_sem = xSemaphoreCreateBinary();
configASSERT(g_send_sem != NULL);

// Create flush timer
g_flush_timer = xTimerCreate("flush", pdMS_TO_TICKS(FLUSH_TIMEOUT_MS),
                             pdFALSE, NULL, flush_timer_cb);

// Create send task
ble_gatt_start_send_task();

// Start NimBLE host task
nimble_port_freertos_init(vNimbleHostTask);
}

```

Only call once. All BLE work is driven by NimBLE host task's event loop.

6.4 on_sync() — Sync Callback and start_scan()

After protocol stack syncs, `on_sync()` is automatically called, which calls `start_scan()` internally to start scanning nearby slaves. `ble_gap_disc()` starts active scanning with all default parameters, `BLE_HS_FOREVER` means continuous scanning without stopping. Scan results are returned via `gap_event_cb()` callback; after matching "ESP-IDF", automatically initiates connection.

```

static void on_sync(void)
{
    ESP_LOGI(TAG, "Host ready, starting scan");
    g_state = STATE_IDLE;
    // Start BLE scan
    start_scan();
}

```

```

void start_scan(void)
{
    // Already scanning
    if (g_state == STATE_SCANNING) return;

    uint8_t own_addr_type;
    // Infer address type
    if (ble_hs_id_infer_auto(0, &own_addr_type) != 0) {
        ESP_LOGE(TAG, "ble_hs_id_infer_auto failed");
        return;
    }
    // Default scan params
    struct ble_gap_disc_params params = {
        .passive = 0, .filter_duplicates = 0, .limited = 0,
        .itvl = 0, .window = 0,
    };
    // Continuous scanning
    int rc = ble_gap_disc(own_addr_type, BLE_HS_FOREVER,
                        &params, gap_event_cb, NULL);

    if (rc != 0) {
        ESP_LOGE(TAG, "ble_gap_disc failed; rc=%d", rc);
        return;
    }

    g_state = STATE_SCANNING;
    ESP_LOGI(TAG, "Scanning for \"%s\" ...", TARGET_DEVICE_NAME);
}

```

6.5 gap_event_cb() — GAP Event Callback

`gap_event_cb()` is the master's central dispatcher, all BLE GAP events are dispatched and handled here:

Event Type	Handling Logic
<code>BLE_GAP_EVENT_DISC</code>	Device discovered → extract device name from advertising packet → name matches <code>"ESP-IDF"</code> then cache address and initiate connection
<code>BLE_GAP_EVENT_CONNECT</code>	Connection success → record connection handle → negotiate MTU → start GATT service discovery
<code>BLE_GAP_EVENT_MTU</code>	MTU negotiation complete → log
<code>BLE_GAP_EVENT_DISCONNECT</code>	Disconnected → flush pending RX buffer → reset all handles → return to scan state
<code>BLE_GAP_EVENT_NOTIFY_RX</code>	Received slave notification → data accumulated to RX buffer → reset 50ms flush timer

```

int gap_event_cb(struct ble_gap_event *event, void *arg)
{
    switch (event->type) {

```



```

// Device discovered
case BLE_GAP_EVENT_DISC: {
    struct ble_gap_disc_desc *d = &event->disc;
    char name[32] = {0};
    // Extract device name
    get_device_name(d, name, sizeof(name));
    if (name[0])
        ESP_LOGI(TAG, "Device: %-15s [%s] RSSI:%d",
                 name, addr_str(&d->addr), d->rss);
    // Target matched
    if (strcmp(name, TARGET_DEVICE_NAME) == 0) {
        ESP_LOGI(TAG, "*** Target \"%s\" found, connecting ***",
                 TARGET_DEVICE_NAME);
        // Cache target address
        memcpy(&g_target_addr, &d->addr, sizeof(ble_addr_t));
        g_target_addr_type = d->addr.type;
        g_target_cached = true;
        // Initiate connection
        ble_central_connect(&g_target_addr, g_target_addr_type);
    }
    return 0;
}

// Connection event
case BLE_GAP_EVENT_CONNECT:
    // Connected
    if (event->connect.status == 0) {
        // Save connection handle
        g_conn_hdl = event->connect.conn_handle;
        ESP_LOGI(TAG, "Connected (conn_handle=%d)", g_conn_hdl);
        // Exchange MTU
        ble_gattc_exchange_mtu(g_conn_hdl, NULL, NULL);
        // Discover GATT services
        discover_service();
    } else {
        ESP_LOGE(TAG, "Connection failed (status=%d)",
                 event->connect.status);
        g_state = STATE_IDLE;
        // Restart scan
        start_scan();
    }
    return 0;
case BLE_GAP_EVENT_MTU:
    ESP_LOGI(TAG, "MTU negotiated: %d", event->mtu.value);
    return 0;

// Disconnected
case BLE_GAP_EVENT_DISCONNECT:
    ESP_LOGI(TAG, "Disconnected (reason=%d), re-scanning...",
             event->disconnect.reason);
    // Flush pending buffer
    if (g_rx_len > 0) flush_timer_cb(NULL);
    // Reset all handles

```

```

g_conn_hdl = g_svc_start = g_svc_end = 0;
g_notify_hdl = g_write_hdl = 0;
g_state = STATE_IDLE;
// Restart scan
start_scan();
return 0;

// Notification received
case BLE_GAP_EVENT_NOTIFY_RX: {
    struct os_mbuf *om = event->notify_rx.om;
    // Notification data length
    uint16_t om_len = OS_MBUF_PKTLEN(om);
    // Flush before overflow
    if (g_rx_len + om_len > sizeof(g_rx_data) - 1) {
        ESP_LOGW(TAG, "RX overflow (%d+%d)", g_rx_len, om_len);
        flush_timer_cb(NULL);
    }
    uint16_t copied;
    // Copy data to buffer
    if (ble_hs_mbuf_to_flat(om, g_rx_data + g_rx_len,
                          sizeof(g_rx_data) - g_rx_len - 1,
                          &copied) != 0)
        return 0;
    g_rx_len += copied;
    // Reset flush timer
    xTimerReset(g_flush_timer, 0);
    return 0;
}

default:
    return 0;
}
}

```

6.6 GATT Discovery Chain — Callback Chain Calls

Master discovers slave's GATT service structure through async callback chain—each step completion automatically triggers next step via callback, going all the way to the bottom:

discover_service()	← Step 1: Find primary service 0xFFFF0
→ gatt_disc_svc_cb()	← Found, record handle range
→ discover_notify_characteristic()	← Step 2: Find notify characteristic 0xFFFF1
→ gatt_disc_chr_notify_cb()	← Found, record handle
→ discover_notify_cccd()	← Step 3: Find CCCD descriptor (0x2902)
→ gatt_disc_dsc_cb()	← Found, write 0x0001 to enable notify
→ enable_notifications()	← Step 4: Notify enabled
→ discover_write_characteristic()	← Step 5: Find write characteristic 0xFFFF2
→ gatt_disc_chr_write_cb()	← Found, record handle
→ mark_ready()	← All ready! Release semaphore

Any step failing to find corresponding UUID logs an error and stops. `enable_notifications()` writes `{0x01, 0x00}` (i.e. 0x0001 little-endian) to CCCD, telling slave to enable notifications.

```

static void enable_notifications(uint16_t cccd_handle)
{
    // 0x0001 = Enable notifications
    uint8_t val[2] = {0x01, 0x00};
    // Write CCCD
    if (ble_gattc_write_flat(g_conn_hdl, cccd_handle, val, sizeof(val),
                            NULL, NULL) != 0) {
        ESP_LOGE(TAG, "CCCD write failed");
        return;
    }
    ESP_LOGI(TAG, "Notification enabled (CCCD=0x%04x)", cccd_handle);

    // Continue pipeline: discover write char
    ble_uuid16_t chr_uuid = BLE_UUID16_INIT(BLE_WRITE_CHR_UUID);
    g_state = STATE_DISCOVER_WRITE_CHR;
    ble_gattc_disc_chrs_by_uuid(g_conn_hdl, g_svc_start, g_svc_end,
                                &chr_uuid.u, gatt_disc_chr_write_cb, NULL);
}

static void mark_ready(void)
{
    // Mark ready
    g_state = STATE_READY;
    ESP_LOGI(TAG, "GATT discovery complete, ready");
    ESP_LOGI(TAG, "  Notify:0x%04x  write:0x%04x", g_notify_hdl, g_write_hdl);
    // Give semaphore
    xSemaphoreGive(g_send_sem);
}

```

6.7 vSendTask() — Periodic Send Task

`vSendTask` is an independent background task running in infinite loop. First time blocks on `xSemaphoreTake()` waiting for GATT discovery completion semaphore; subsequent times semaphore take returns due to 5-second timeout, achieving every 5 seconds auto-send. `ble_gattc_write_flat()` writes data to slave's 0xFFF2 characteristic.

```

static void vSendTask(void *param)
{
    // Message to send
    const char *msg = "Hello from ESP32 Master";
    while (1) {
        // wait for semaphore (first=GATT ready, subsequent=periodic trigger)
        if (xSemaphoreTake(g_send_sem,
                          pdMS_TO_TICKS(SEND_INTERVAL_MS)) == pdTRUE) { }

        // Ready & handle valid
        if (g_state == STATE_READY && g_write_hdl != 0) {
            // write to slave
            int rc = ble_gattc_write_flat(g_conn_hdl, g_write_hdl,
                                          msg, strlen(msg), NULL, NULL);

            // Send OK
            if (rc == 0) ESP_LOGI(TAG, "Sent: %s", msg);
        }
    }
}

```

```

        // Send failed
    else ESP_LOGE(TAG, "write failed; rc=%d", rc);
    }
}
}

```

6.8 flush_timer_cb() — RX Accumulation Flush

BLE notifications from slave may arrive in multiple packets. `flush_timer_cb()` mechanism: each time data received, append to `g_rx_data` buffer and reset 50ms timer; if no new data within 50ms, message reception is considered complete, print complete content at once.

```

void flush_timer_cb(TimerHandle_t timer)
{
    // No data, skip
    if (g_rx_len == 0) return;
    // Add terminator
    g_rx_data[g_rx_len] = '\0';
    // Print complete message
    ESP_LOGI(TAG, "Slave(%d bytes): %s", g_rx_len, g_rx_data);
    // Reset buffer
    g_rx_len = 0;
}

```

7. Operation Flow

7.1 Build and Flash Flow

ESP-IDF Flash Flow: See `ESP-IDF → 1. Before Development → 3. Build and Flash`.

Two ways to test master: ① Companion another ESP32-S3 flashed with BLE Slave routine for dual-device communication (section 7.2); ② Companion phone Bluetooth debugging assistant App to verify slave-side data (section 7.3).

7.2 Testing Method 1: Dual-Board Test

1. First power on Slave board, confirm serial outputs `Advertising as "ESP-IDF" ...` indicating advertising started.
2. Power on Master board, serial starts scanning and lists all nearby BLE devices. Intermixing non-target devices is normal.

```

I (483) BLE_MASTER: Scanning for "ESP-IDF" ...
I (483) main_task: Returned from app_main()
I (733) BLE_MASTER: Device: EDIFIER BLE [CC:14:BC:02:85:7E] RSSI:-52
I (833) BLE_MASTER: Device: Baseus BowieE3 [41:AA:67:1B:51:AB] RSSI:-64
I (913) BLE_MASTER: Device: midea [D8:34:1C:C2:48:FB] RSSI:-91
I (973) BLE_MASTER: Device: midea [D8:34:1C:C2:48:FB] RSSI:-90
I (973) BLE_MASTER: Device: midea [D8:34:1C:C2:48:FB] RSSI:-90
I (1203) BLE_MASTER: Device: EH-MC16-CZ [18:14:6C:55:48:D5] RSSI:-92
I (1233) BLE_MASTER: Device: NG0c8Mm/UyaywzgMJEQv5GUMc [62:56:3A:40:79:D0] RSSI:-83
I (1243) BLE_MASTER: Device: midea [D8:34:1C:C2:48:FB] RSSI:-94
I (1263) BLE_MASTER: Device: NGMOyoibqRfZvfKpdpX6Vxi34 [40:02:2F:70:02:FA] RSSI:-68

```

3. Master scans `ESP-IDF` and automatically connects. Serial outputs in sequence: `Target "ESP-IDF" found` → `Connected` → `MTU negotiated` → `GATT discovery chain (Service 0xFFFF0 → Notify char 0xFFFF1 → CCCD → Notification enabled → write char 0xFFFF2) → GATT discovery complete - ready`.

```

I (2363) BLE_MASTER: *** Target "ESP-IDF" found - connecting ***
I (2373) NimBLE: GAP procedure initiated: connect;
I (2373) NimBLE: peer_addr_type=0 peer_addr=
I (2373) NimBLE: a4:cb:8f:c6:b8:36
I (2383) NimBLE: scan_itvl=16 scan_window=16 itvl_min=24 itvl_max=40 latency=0 supervision_timeout=256 min_ce_len=0 max_ce_len=0 own_addr_type=0
I (2393) NimBLE:
I (2503) BLE_MASTER: Connected (conn_handle=1)
I (2503) NimBLE: GATT procedure initiated: exchange mtu

```

```

I (2503) NimBLE: GATT procedure initiated: discover service by uuid; uuid=
I (2503) NimBLE: 0xffff0
I (2503) NimBLE:
I (2553) BLE_MASTER: MTU negotiated: 256
I (2553) BLE_MASTER: MTU negotiated: 256
I (2753) BLE_MASTER: Service 0xFFFF0 [0x000E-0x0013]
I (2753) NimBLE: GATT procedure initiated: discover characteristics by uuid;
I (2753) NimBLE: start_handle=14 end_handle=19 uuid=
I (2753) NimBLE: 0xffff1
I (2753) NimBLE:
I (2953) BLE_MASTER: Notify char 0xFFFF1 (val=0x0010)
I (2953) NimBLE: GATT procedure initiated: discover all descriptors;
I (2953) NimBLE: chr_val_handle=16 end_handle=19

```

```

I (3103) BLE_MASTER: Notification enabled (CCCD=0x0010)
I (3103) NimBLE: GATT procedure initiated: discover characteristics by uuid;
I (3113) NimBLE: start_handle=14 end_handle=19 uuid=
I (3113) NimBLE: 0xffff2
I (3123) NimBLE:
I (3303) BLE_MASTER: Write char 0xFFFF2 (val=0x0013)
I (3403) BLE_MASTER: GATT discovery complete - ready
I (3403) BLE_MASTER: Notify:0x0010 Write:0x0013
I (3403) NimBLE: GATT procedure initiated: write;
I (3403) NimBLE: att_handle=19 len=23

```

4. After GATT discovery complete, Master immediately writes `"Hello from ESP32 Master"` via `0xFFFF2`.
Serial outputs `Sent: Hello from ESP32 Master`.
5. Slave pushes notification every 6 seconds. Master serial outputs `Slave(26 bytes): Hello from ESP32`

`sSlave [N]` (first received count N depends on how long slave has been running, not necessarily 0).

```
I (1753) BLE_MASTER: Sent: Hello from ESP32 Master
I (6703) BLE_MASTER: Slave(26 bytes): Hello from ESP32 Slave [0]
I (6763) NimBLE: GATT procedure initiated: write;
I (6763) NimBLE: att_handle=19 len=23
```

6. Both message types output alternately and independently, proving bidirectional data channel works.

7.3 Testing Method 2: With Phone Bluetooth Debugging Assistant

For only one development board, flash Master routine and pair with phone Bluetooth debugging assistant App to verify write and notification functions from phone side.

Common Apps: nRF Connect for Mobile (recommended), LightBlue, EFR Connect, BLE Scanner, all free.

Steps:

1. Power on Master board, serial outputs `Scanning for "ESP-IDF" ...` indicating scan started.

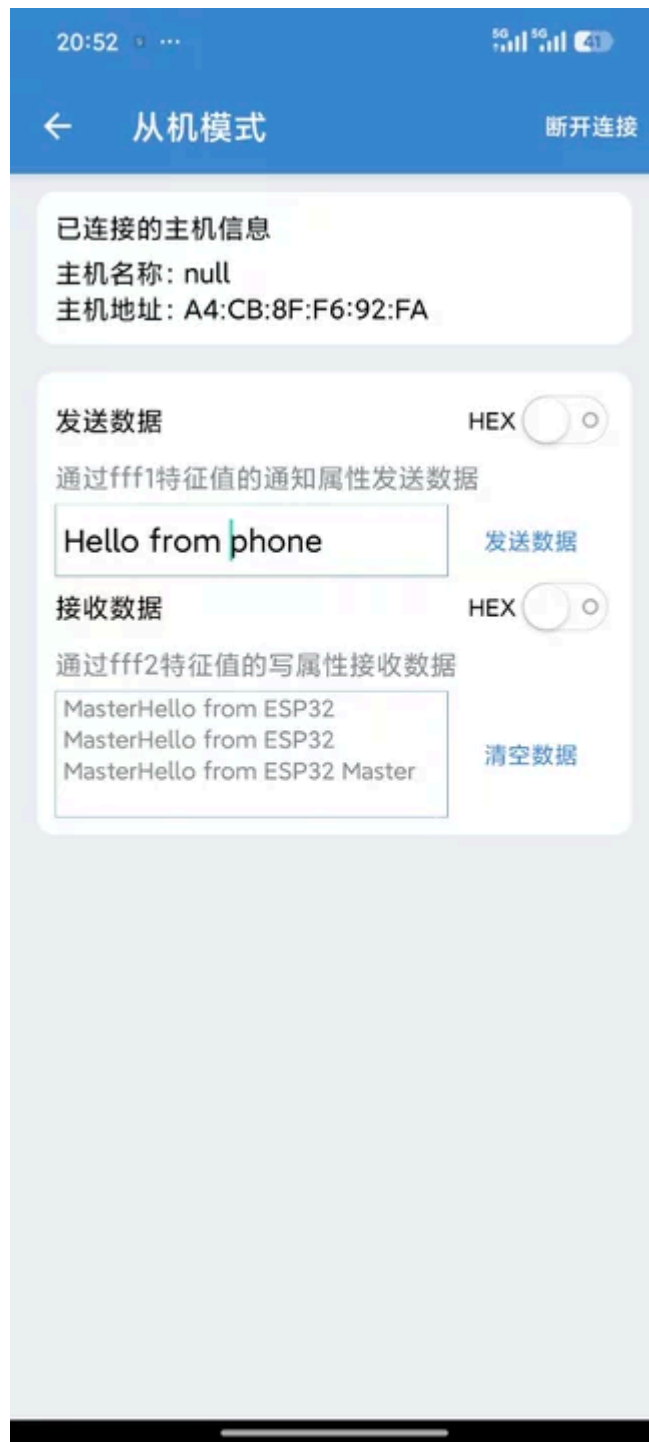
```
I (483) BLE_MASTER: Scanning for "ESP-IDF" ...
I (483) main_task: Returned from app_main()
I (733) BLE_MASTER: Device: EDIFIER BLE [CC:14:BC:02:85:7E] RSSI:-52
I (833) BLE_MASTER: Device: Baseus BowieE3 [41:AA:67:1B:51:AB] RSSI:-64
I (913) BLE_MASTER: Device: midea [D8:34:1C:C2:48:FB] RSSI:-91
I (973) BLE_MASTER: Device: midea [D8:34:1C:C2:48:FB] RSSI:-90
I (973) BLE_MASTER: Device: midea [D8:34:1C:C2:48:FB] RSSI:-90
I (1203) BLE_MASTER: Device: EH-MC16-CZ [18:14:6C:55:48:D5] RSSI:-92
I (1233) BLE_MASTER: Device: NG0c8Mm/UyaywzgMJEQv5GUWc [62:56:3A:40:79:D0] RSSI:-83
I (1243) BLE_MASTER: Device: midea [D8:34:1C:C2:48:FB] RSSI:-94
I (1263) BLE_MASTER: Device: NGMOyoibqRfZvfKpdpX6Vxi34 [40:02:2F:70:02:FA] RSSI:-68
```

2. Master needs to find slave advertising as `"ESP-IDF"` to connect—so need to simultaneously run a BLE Slave (another board or another ESP32), or use phone App to simulate slave advertising (e.g., nRF Connect's Advertiser mode, advertising name set to `"ESP-IDF"`).

```
I (14063) BLE_MASTER: *** Target "ESP-IDF" found - connecting ***
I (14073) NimBLE: GAP procedure initiated: connect;
```

3. After Master connects to slave, serial outputs complete GATT discovery log (`Service 0xFFFF0 → Notify char 0xFFFF1 → CCCD → write char 0xFFFF2`), finally displays `GATT discovery complete, ready`.

4. Master automatically writes to slave every 5 seconds `"Hello from ESP32 Master"`. If slave receives normally, slave serial prints `Master(23 bytes): Hello from ESP32 Master`.



- Most phones don't support slave mode, slave mode mostly only used for receiving master's information, so dual-board test is recommended.

7.4 Normal Communication Log (Dual-Board Test)

Field	Example	Meaning
I	I	INFO level log
BLE_MASTER	BLE_MASTER	Log tag
Device:	Device:	Scanned device name
Sent:	Sent:	Write operation content sent by master

Field	Example	Meaning
Slave(N bytes):	Slave:	Notification data from slave

Below is actual measured log (NimBLE protocol stack internal debug info filtered, only valid output with `BLE_MASTER` tag). Scan phase lists all nearby BLE devices; many non-target devices like earphones, home appliances mixed in is normal—Master only initiates connection when matching `"ESP-IDF"`.

```
I (483) BLE_MASTER: Scanning for "ESP-IDF" ...
I (483) main_task: Returned from app_main()
I (733) BLE_MASTER: Device: EDIFIER BLE      [CC:14:BC:02:85:7E] RSSI:-52
I (833) BLE_MASTER: Device: Baseus BowieE3   [41:AA:67:1B:51:AB] RSSI:-64
I (913) BLE_MASTER: Device: midea            [D8:34:1C:C2:48:FB] RSSI:-91
... (other non-target devices omitted) ...
I (2363) BLE_MASTER: Device: ESP-IDF         [A4:CB:8F:C6:B8:36] RSSI:-36
I (2363) BLE_MASTER: *** Target "ESP-IDF" found - connecting ***
I (2503) BLE_MASTER: Connected (conn_handle=1)
I (2553) BLE_MASTER: MTU negotiated: 256
I (2753) BLE_MASTER: Service 0xFFFF0 [0x000E-0x0013]
I (2953) BLE_MASTER: Notify char 0xFFFF1 (val=0x0010)
I (3053) BLE_MASTER: CCCD at 0x0010
I (3053) BLE_MASTER: Notification enabled (CCCD=0x0010)
I (3253) BLE_MASTER: Write char 0xFFFF2 (val=0x0013)
I (3353) BLE_MASTER: GATT discovery complete - ready
I (3353) BLE_MASTER:   Notify:0x0010 write:0x0013
I (3353) BLE_MASTER: Sent: Hello from ESP32 Master
I (3203) BLE_MASTER: Slave(26 bytes): Hello from ESP32 Slave [1]
```

7.5 Edge Cases

- **Only Master powered (no slave advertising):** Master continuously scans. When Slave comes online, auto-connects.
- **Slave powers off first:** Master detects disconnection then automatically returns to scan state. When Slave repowers, auto-reconnects.
- **Slave changed but same name:** As long as advertising name still `"ESP-IDF"`, MAC address change doesn't affect connection.

8. Common Problems

Symptom	Cause	Solution
Master can't find slave	Slave not powered / advertising name mismatch	Confirm Slave flashed and powered, advertising name is <code>"ESP-IDF"</code>
Connection success but no data	GATT discovery chain interrupted (UUID mismatch)	Confirm Slave service UUID is <code>0xFFFF0</code> , Master side UUID matches

Symptom	Cause	Solution
Notifications not received	CCCD not enabled / notify characteristic handle is 0	Check slave 0xFFFF1 characteristic flags include <code>BLE_GATT_CHR_F_NOTIFY</code>
Write fails	Write characteristic handle is 0 / connection disconnected	Confirm slave 0xFFFF2 characteristic flags include <code>BLE_GATT_CHR_F_WRITE</code>
<code>BLE_HS_EDONE</code> error	Service/characteristic/descriptor with corresponding UUID not found	Check slave GATT service table definition, UUID matches Master macro definition
Doesn't reconnect after disconnection	<code>start_scan()</code> re-entry protection causing state stuck	Confirm <code>BLE_GAP_EVENT_DISCONNECT</code> correctly sets <code>g_state = STATE_IDLE</code>
NVS init error	NVS partition corrupted or unpartitioned	IDE flash with "Erase flash" checked or run <code>idf.py erase-flash</code>